Epicourier

The meal planner that shows its working — and lets you correct it.

Group 12
Wenbo Li · Sihao Liu · Andy Yu · Jeffery Zheng
CSC 510 · Project 2 · Fall 2026

MISSION — WHY / WHAT / SO WHAT

Why. Epicourier already recommends recipes from your pantry, and the match percentage it prints in bold is written by a language model. **No line of our code computes it.** The response schema has no field that could hold a per-ingredient contribution, so **0 of 300** returned scores can be broken down — not an estimate, a property of the schema. A user who disagrees has nothing to point at, and the team cannot tell whether a change improved the ranking or merely moved it.

What. We replaced the number with a **receipt**: ranking computed by a pure function against a published table and printed as lines that add up to the total — each naming the pantry item, its date, and the points it contributed. Correct any line and the order moves. The model keeps the sentence it is good at and loses the authority to invent the score.

So what. The ranking became measurable, so it became improvable — each claim stated as a threshold *before* the code existed, and each met.

0 → 100%	1,579	50.1%	5
scores that decompose into line items	tests run green 1,504 Jest + 75 pytest	of the codebase is test code	CI gates per push, none faked

MILESTONES — BEFORE · NOW · FUTURE

BEFORE · P1a

- **Reverse-engineer 20 use cases.** 4 LLMs as independent analysts; every claim checked against source, 5 false ones corrected.
- **Write and run our own suite.** Web 32, backend 18 — authored by us, not inherited.
- **Adversarial pass.** 69 cases; 18 failures triaged to 17 distinct defects, all re-confirmed.
- **Correct our own findings.** Three reversals published, including a headline claim we weakened ourselves.

NOW · P2 — shipped

- **M0 · risk-killer eval.** Thresholds and baselines; 500-cell corpus from the real dataset; a corrupted case CI must reject.
- **M1 · deterministic scoring.** Pure function — no model, no clock; expired stock unusable, zero stays zero.
- **M2 · receipt panel.** Every line visible and correctable; edits persist and re-rank.
- **M3 · pantry delta.** missing = max(required − usable, 0); preview writes nothing, confirm is idempotent in the database.

FUTURE · P3

- **Food safety in code.** "Never recommend expired" lives in a prompt string today. Move it into the scorer with a test that fails on regression.
- **Per-purchase lots.** The schema sums per location, so "which batch" cannot be answered. Add the table, migrate.
- **Verification gate.** Check every ingredient an explanation asserts against the 448-row catalogue.
- **Does it change behaviour?** A within-subject study. Labelled a hypothesis, not a need.

THE SYSTEM RUNNING — REAL CAPTURES, CLEAN CHECKOUT, NO KEY, NO NETWORK

```
$ cd web && npx jest --silent
Test Suites: 1 skipped, 84 passed, 84 of 85 total
Tests:       1 skipped, 1503 passed, 1504 total

$ cd backend && uv run pytest tests/p2 -q      # in CI, offline
75 passed in 2.31s

$ python3 p2/eval/run_eval.py
ok    hand-computed: 3 receipts, no violations
ok    scorer-vs-hand-computed: the real scorer matches every hand-written value
ok    corrections: top-up, expiry, unit and relocation edits all move the score as the rules say
ok    corpus: 500 cells scored, 6120 receipt lines, every receipt adds up; 25 cells x 20 shuffles order-blind
ok    shuffle-invariance: 20 inventory orderings produced identical usable quantities
M0 hypotheses (see p2/eval/HYPOTHESIS.md)
  H1  MET    500/500 = 100.0%  (threshold 100%)  every displayed total is the sum of its displayed parts
      baseline: 0/300 (old path cannot decompose at all)
  H2  MET    25/25 = 100.0%  (threshold 100%)  identical input produces identical output
      baseline: X — NOT COLLECTED
  H3  MET    4/4 = 100.0%  (threshold 100%)  a correction moves the score the way the rules say
      baseline: n/a — no correction loop in the old path
PASS
```

What this output is claiming. The expectations were written by hand from the frozen contract *before any scorer existed*; the scorer was then written independently against the same contract. **They agree on every value** — three totals, nine subtotals, fifteen receipt lines. Two independent derivations matching is evidence; a function compared with itself is not, which is why a deliberately corrupted case is checked first and CI fails if the suite ever accepts it.

WHAT WORKS TODAY — WHAT A PROJECT 3 TEAM INHERITS ON DAY ONE

- **A receipt that adds up** — line points sum to the total on all 500 cells.
- **A frozen four-module contract** — schema, TypeScript and Pydantic, tests fail on drift.
- **An evaluation watched failing** — a corrupted case CI is required to reject.
- **Correctable inventory** — edits persist, re-rank, and relocation changes nothing.

STAKEHOLDERS

- **Household cook** — wants dinner decided, not an audit.
- **The co-resident who never installed it** — eats the result, corrects nothing.
- **The person with the allergy** — a wrong substitution is not a ranking error.
- **Recipe author** — summarised by an LLM, never asked.
- **On-call maintainer at 3 a.m.** — needs a number they can recompute.
- **Next semester's inheriting team** — the reader of this poster.

STACK — AND WHY IT MATTERS TO YOU

Next.js 15 · TypeScript · Tailwind · FastAPI · Python 3.11 · Supabase / Postgres · Gemini 2.5 Flash · Jest · pytest · Ruff · ESLint · GitHub Actions

Why inherit this one: the scoring core is a pure Python function with **no network, no key and no clock** — run the whole evaluation on a fresh clone in one command, before installing anything. The contract between the four modules is frozen in JSON Schema with `additionalProperties:false`, so a change that breaks another module fails a test instead of surprising you in week three.

SCREENS

Receipt panel — each line names the item, its date and the points it added; the total is their sum.
Shopping preview — 500 g needed, 200 g on hand, 300 g proposed; nothing is written until you confirm.

*The **3 min 41 s walk-through** shows every feature running — scan the Walkthrough code below. Still shots of the UI are pending credentials; the terminal panel opposite is a real, unedited capture.*

```
Receipt · recipe 101 · as_of 2026-10-10
 chicken 500 g req · 350 usable  missing +0.0
 rice    200 g req · 300 usable  ok      +1.0
 basil     0 g req ·   0 usable  expired +1.0
 expiry urgency, fridge lot              +1.0
```

FIND US

Walkthrough
youtu.be/NPHCkjngmlo

Self-assessed rubric 79/96 — every row scored against evidence that a spot-check can open; the one row still at 0 says why. Table in `p2/rubric/self-assessment.md`.

Teaser 28 s · Walk-through 3 min 41 s — both within the required ranges. Baseline **X not collected**: it needs 30 live model calls we have not made.